



كلية العلوم ببزرت
Faculté des Sciences de Bizerte



FASCICULE DE COURS (VOLET PRATIQUE)

MACHINE LEARNING2

numpy, pandas, matplotlib & seaborn

2^{ème} année Mastère pro Data Science



DR. TAOUFIK BEN ABDALLAH



MAÎTRE-ASSISTANT À LA FS-BIZERTE
UNIVERSITÉ DE CARTHAGE



taoufik.benabdallah@fsb.ucar.tn



Plan de la présentation

PARTIE I Calcul scientifique avec  NumPy

PARTIE II Exploration des données avec  pandas

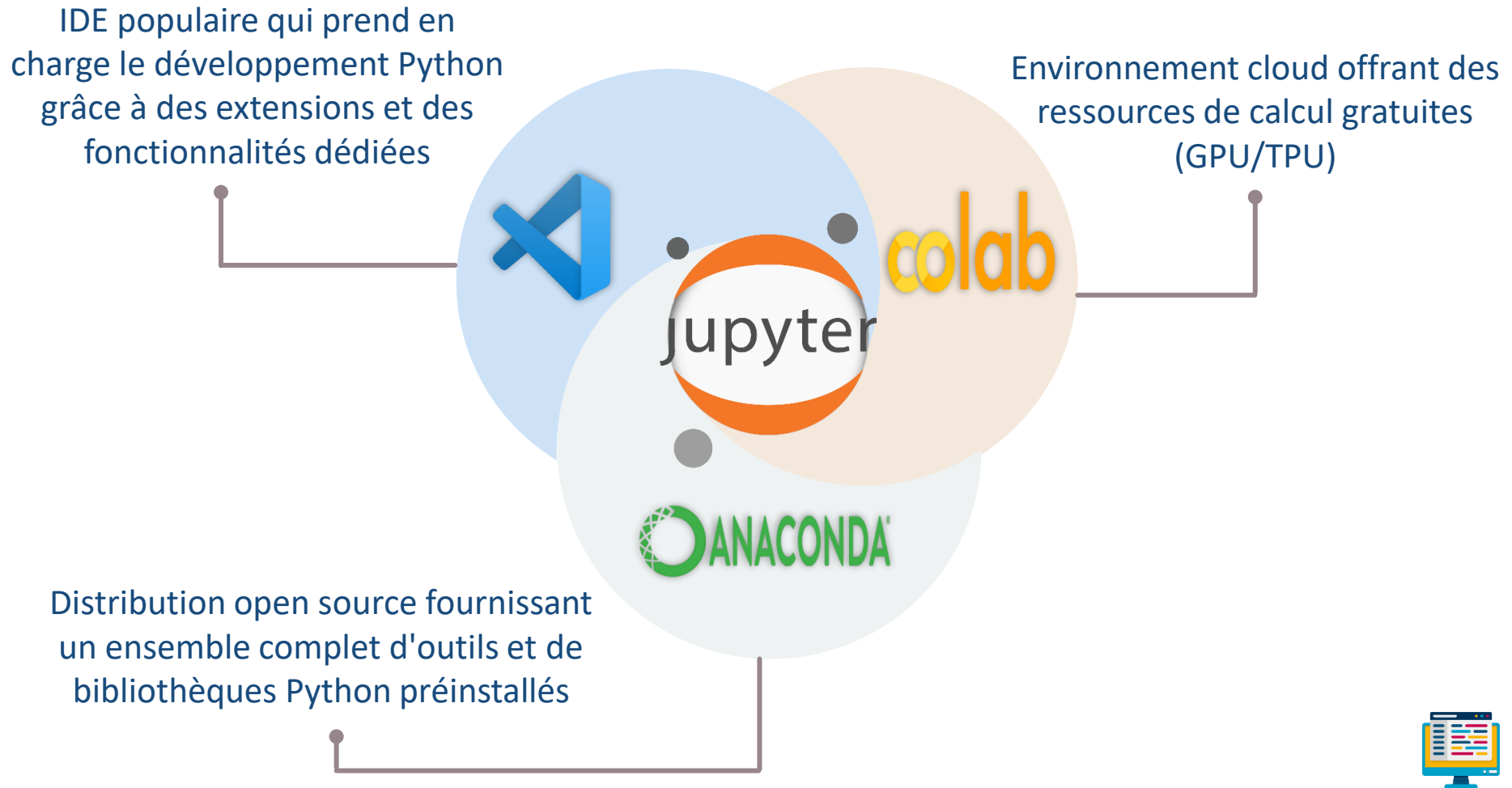
PARTIE III Visualisation graphique avec *matplotlib*  & *seaborn* 



Environnement de développement intégré (IDE)



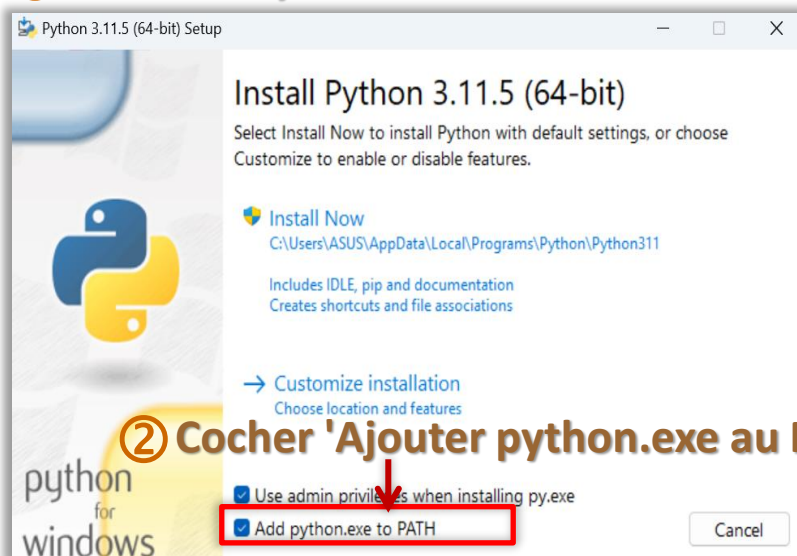
Jupyter notebook = IDE + outil de présentation



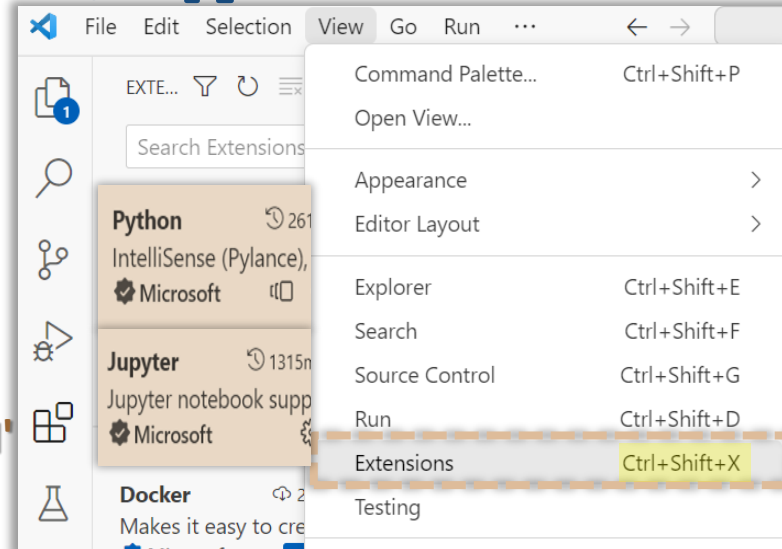


③ Ouvrir Visual Studio Code & Installer les 2 extensions : Python / Jupyter Notebook de Microsoft

① Installer Python3.x



② Cocher 'Ajouter python.exe au PATH'



④ Installer les bibliothèques nécessaires!

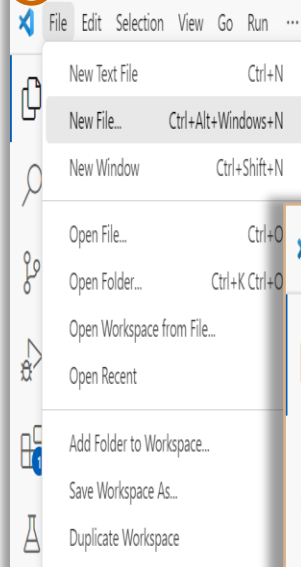
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER powershell ell + v [trash] ... ^ X
PS C:\Users> pip install numpy pandas matplotlib seaborn
pip install -U scikit-learn
```

Création d'un nouveau Notebook avec

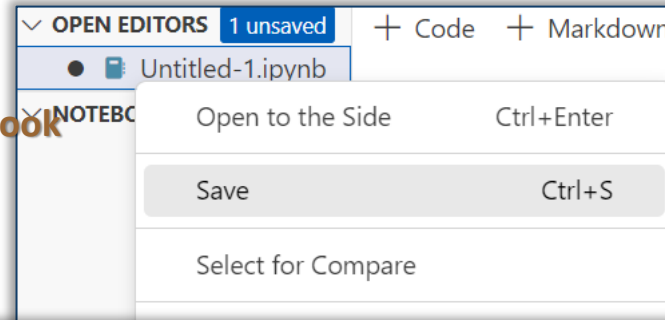
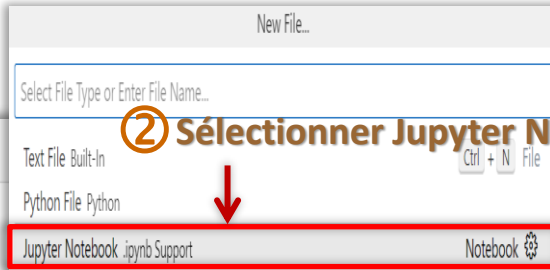


③ File → Save → Sélectionner le chemin et renommer le fichier!

① File → New File



② Sélectionner Jupyter Notebook



← Fichier d'extension .ipynb

Espace de création de code



PARTIE I

CALCUL SCIENTIFIQUE AVEC



Création des tableaux & propriétés

- **Importation de numpy :** `import numpy as np`

- **Création des Tableaux ndarray**

```
T1=np.arange(1,6) [1 2 3 4 5]
```

```
T2=np.array([[10,20],[30,50.2],[60,70]])  
[[10. 20. ]  
 [30. 50.2]  
 [60. 70. ]]
```

#Tableaux régulièrement espacés sur un intervalle spécifié

```
T3=np.linspace(2,10,5) [ 2.  4.  6.  8. 10.]
```

- **Conversion de type** `T3=T3.astype("int64")`

`T.shape`

`T.ndim`

`T.size`

`len(T)`

`T.dtype`



Accès et Slicing ndarray

● Accès indexée

```
T[0,:] Équivalent à T[0]  [10 20 10 7]
T[-1]      [21 22 5 13]
T[:,1]     [20 11 22]
T[1,1]     11
T[0:2,1:]  [[20 10 7]
             [11 50 44]]
```

```
T=
[[10 20 10 7]
 [ 8 11 50 44]
 [21 22 5 13]]
```

● Accès conditionnelle & Modification

```
T[T>10]  [20, 11, 50, 44, 21, 22, 13]  ndarray de 1D!
```

```
T>10
[[False, True, False, False]
 [False, True, True, True]
 [ True, True, False, True]]
```

```
T1=np.where(T > 30, -1, T)  T1?
```

```
T2=np.where(T > 30, np.where(T >=50, -2, -3), T)  T2?
```

Opérations sur les tableaux ndarray (1/3)

● Modification de forme

`T1=np.reshape(T, [2,3])` ❌

`T1=np.resize(T, [2,3])` `[[ 7 14 10]`
 `[ 2 7 14]]`

`T2=np.ravel(T1)` `[ 7 14 10 2 7 14]`

`T=`
`[ 7 14 10 2]`

● Calcul de la somme des éléments np.sum(...)

`np.sum(T)` 85

`np.sum(T, axis=0)` `[ 20 50 15]`

La somme des éléments par lignes (axis=0) pour chaque colonne

`np.sum(T, axis=1)` `[ 40 45]`

La somme des éléments par colonnes (axis=1) pour chaque ligne

`T=`
`[[ 10 20 10]`
 `[ 10 30 5]]`

Opérations sur les tableaux ndarray (2/3)

- Calcul d'indice de maximum `np.argmax(...)`

```
np.argmax(T)    4
```

```
np.argmax(T, axis=0)  [ 0  1  0]
```

```
np.argmax(T, axis=1)  [ 1  1]
```

T=

```
[[ 10  20  10]
 [ 10  30  5]]
```

- `np.max(...)` / `np.mean(...)` / `np.std(...)` /

- Concaténation `np.concatenate(...)` & Insertion `np.insert(...)`

Par défaut `axis=0`

```
T2=np.concatenate([T, T1], axis=1) [[10  20  10  22  44]
                                     [10  30  5  55  33]]
```

Par défaut `axis=None`

```
T3=np.insert(T, 1, [-1, -1], axis=1) [[10 -1  20  10]
                                       [10 -1  30  5]]
```

T1=

```
[[ 22  44]
 [ 55  33]]
```

#axis=None : Convertir le tableaux en 1D et insérer l'élément dans la position donnée!

Opérations sur les tableaux ndarray (3/3)

● Décomposition d'un tableau `np.split(...)`

#Décomposer **T** en **3** tableaux : le premier tableau comprend les colonnes d'indices 0 et 1, le deuxième tableau comprend la colonne d'indice 2, et le dernier tableau comprend les colonnes restantes de **T** ?

T=

```
[[ 10  20  10  7  11]
 [ 10  30  5   6  12]]
```

Par défaut `axis=0`

```
A= np.split(T, [2, 3], axis=1)
↑
list
```

`[0:2[ / [2:3[ / [3:]`

```
[array([[10, 20], [10, 30]]),
 array([[10], [ 5]]), array([[ 7, 11],
 [ 6, 12]])]
```

`A[1].shape ? (2,1)`

● Sélection aléatoire des éléments `np.random.choice(...)` 1D!

~~`np.random.choice(T, size=4, replace=False)`~~ → `T.ravel()`

#Sélectionner aléatoirement N=size valeurs à partir du tableau

`replace= False` : échantillonnage sans remplacement /par défaut `size=None, replace=True`

Exercice 1/2

```
arr = np.array([15, 27, 8, 36, 41, 12, 6, 95, 18, 50])
```

1/ Remplacer tous les éléments pairs du tableau **arr** par -1

```
arr[arr%2==0]=-1
```

#ou bien

```
arr=np.where(arr%2==0,-1,arr)
```

```
print(arr) #[15 27 -1 -1 41 -1 -1 95 -1 -1]
```

2/ Modifier la taille de **arr** en (2,5), puis concaténer-le par colonne avec le tableau **ndarray** suivant [10, -10]

```
arr=arr.reshape(2, 5)
```

```
concatenated_arr = np.concatenate([arr,  
                                   np.array([10, -10]).reshape(2,1)], axis=1)
```

```
print(concatenated_arr) #[[ 15 27 -1 -1 41 10]  
                        [-1 -1 95 -1 -1 -10]]
```

Exercice 2/2

3/ Trouver l'indice de l'élément maximum dans le tableau concaténé

```
max_index=np.argmax(concatenated_arr) #8
```

Déterminer le couple d'indices (indice de ligne, indice de colonne) de l'élément ayant la valeur maximale?

```
c_indices=np.unravel_index(max_index, concatenated_arr.shape)
print(c_indices) #(1, 2)
```

4/ Diviser le tableau concaténé par ligne en 2 nouveaux tableaux, **arr1** et **arr2**, chacun de taille (1, 6)

```
arr1,arr2=np.split(concatenated_arr,2, axis=0)
print(arr1, arr2)#[[15 27 -1 -1 41 10]] [[-1 -1 95 -1 -1 -10]]
#ou bien
arr1 = concatenated_arr[0, :-1]
arr2 = concatenated_arr[1, :-1]
print(arr1, arr2)#[15 27 -1 -1 41 10] [-1 -1 95 -1 -1 -10]
```



PARTIE II

EXPLORATION DES DONNÉES AVEC

 pandas

DataFrame!

- Importation de pandas : `import pandas as pd`

- Création d'un DataFrame

```
df=pd.DataFrame([[7,10,'AA',12.2,1],  
                 [20,22,'BC',13.3,0],[10,8,'AA',22.1]],  
                columns=['A1','A2','A3','A4','A5'])
```

DataFrame df

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0
2	10	8	AA	22.1	NaN

columns

index

- × Information & description sur les données

`df.dtypes`

`df.info()`

`df.describe()`



- × Conversion de type

```
df=df.astype({'A5':'float32'})
```


Ecriture & Lecture d'un fichier csv/xlsx

● DataFrame → csv/ xlsx

Valeur par défaut

```
df.to_csv('/content/drive/MyDrive/exemple.csv',  
          Par défaut index=True  
          index=False, sep=',')
```

```
df.to_excel('/content/drive/MyDrive/exemple.xlsx',  
            Par défaut index=True  
            index=False, sheet_name='Sheet1')
```

DataFrame df

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0
2	10	8	AA	22.1	NaN

exemple.csv

Fichier Modifier Affichage

```
A1,A2,A3,A4,A5  
7,10,AA,12.2,1.0  
20,22,BC,13.3,0.0  
10,8,AA,22.1,
```

● Lecture d'un DataFrame à partir d'un fichier csv ou xlsx

```
df=pd.read_csv('/content/drive/MyDrive/exemple.csv', sep=',')
```

```
df=pd.read_excel('/content/drive/MyDrive/exemple.xlsx',  
                 sheet_name=0)
```

exemple					- Excel
Fichier	Accueil	Insertion	Mise en page	Formules	Données
A1	:	✕	✓	fx	A1
1	A1	A2	A3	A4	A5
2	7	10	AA	12,2	1
3	20	22	BC	13,3	0
4	10	8	AA	22,1	
5					
6					
Sheet1					+

Accès et Slicing DataFrame

`df.iloc[:,[1,2]]`

	A2	A3
0	10	AA
1	22	BC
2	8	AA

`df.loc[:,['A1']]`

	A1
0	7
1	20
2	10

`df.iloc[0:2,:]`

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0

`df.loc[0:2,:]`

DataFrame `df`

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0
2	10	8	AA	22.1	NaN

`df.head() / df.tail()` ?

Accès conditionnelle

`df[df['A2']>=10]`

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0

`df[(df['A2']>=10) & (df['A4']>13)]`

	A1	A2	A3	A4	A5
1	20	22	BC	13.3	0.0

Opérations sur DataFrame (1/4)

● Fusion des DataFrames

Par défaut `axis=0`

```
df_c=pd.concat([df,df1], axis=0, ignore_index=True)
```

df_c=

	A1	A2	A3	A4	A5	0	1	2
0	7.0	10.0	AA	12.2	1.0	NaN	NaN	NaN
1	20.0	22.0	BC	13.3	0.0	NaN	NaN	NaN
2	10.0	8.0	AA	22.1	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	33.0	44.0	55.0
4	NaN	NaN	NaN	NaN	NaN	66.0	22.0	11.0

```
df_c=pd.concat([df,df1], axis=1)
```

df_c=

	A1	A2	A3	A4	A5	0	1	2
0	7	10	AA	12.2	1.0	NaN	NaN	NaN
1	20	22	BC	13.3	0.0	33.0	44.0	55.0
2	10	8	AA	22.1	NaN	66.0	22.0	11.0

df=

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0
2	10	8	AA	22.1	NaN

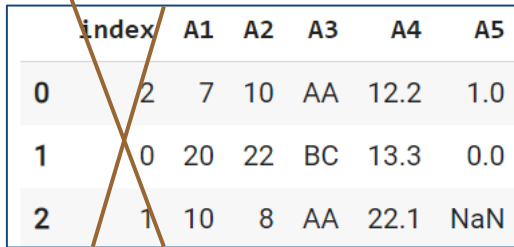
df1=

	0	1	2
1	33	44	55
2	66	22	11

Opérations sur DataFrame (2/4)

● Réinitialisation de l'index des lignes

```
df.reset_index(drop = True, inplace=True)
```



	index	A1	A2	A3	A4	A5
0	2	7	10	AA	12.2	1.0
1	0	20	22	BC	13.3	0.0
2	1	10	8	AA	22.1	NaN

df=

	A1	A2	A3	A4	A5
2	7	10	AA	12.2	1.0
0	20	22	BC	13.3	0.0
1	10	8	AA	22.1	NaN

● Tri sur les données d'un DataFrame

Par défaut ascending=True
(Tri croissant)

```
df.sort_values(by='A2', inplace=True, ascending=True, ignore_index=True)
```

Opérations sur DataFrame (3/4)

● Suppression des lignes/colonnes à partir d'un DataFrame

Par défaut `axis=0`

```
df.drop(labels=['A1', 'A2'], axis=1, inplace=True)
```

```
df.drop(labels=[0, 1], inplace=True)
```

df=

	A1	A2	A3	A4	A5
0	7	10	AA	12.2	1.0
1	20	22	BC	13.3	0.0
2	10	8	AA	22.1	NaN

	A3	A4	A5
2	BC	13.3	0.0

● Modification des noms des colonnes d'un DataFrame

```
df.rename(columns={'A1': 'A', 'A2': 'B', 'A3': 'C'},  
          inplace=True)
```

df=

	A	B	C
0	CCC	12	11
1	CCC	55	33
2	EEE	NaN	66

● Modification des valeurs dans un DataFrame

```
df["B"].replace(44, np.nan, inplace=True)
```

```
df["A"].replace({"DDD": "CCC", "AAA": "EEE"}, inplace=True)
```

Opérations sur DataFrame (4/4)

- Division des données d'un DataFrame en groupes

```
df.groupby(['A'])['C'].max()
```

```
A  
CCC    33  
EEE    66  
Name: C, dtype: int64
```

df=

	A	B	C
0	CCC	12.0	11
1	CCC	55.0	33
2	EEE	NaN	66

- Calcul du nombre d'occurrences dans un DataFrame

```
df['A'].value_counts()
```

```
CCC    2  
EEE    1  
Name: A, dtype: int64
```

- DataFrame → ndarray

```
X= df.iloc[:,2].to_numpy() Équivalent à X= df.iloc[:,2].values #[11 33 66]
```

```
X= df.iloc[:,[2]].to_numpy() ? X.shape #(3, 1)
```

Exercice 1/3

```
biens=np.array([[85,120],[100,200],[85,150],[200,180],  
               [100,420],[300,330],[400,400]])
```

1/ Générer un DataFrame à partir de **biens**, **df_biens**, possédant 2 colonnes, nommées respectivement **surface** et **prix**. Afficher les 5 premières lignes de **df_biens**

```
df_biens= pd.DataFrame(biens,  
                       columns=['surface', 'prix'])  
  
df_biens.head()
```

	surface	prix
0	85	120
1	100	200
2	85	150
3	200	180
4	100	420
5	300	330
6	400	400

2/ Afficher les surfaces des biens ayant un prix supérieur ou égal à 300

```
df_biens[df_biens['prix']>=300].loc[:,['surface']]  
  
df_biens[df_biens['prix']>=300]['surface'] ?
```

	surface
4	100
5	300
6	400

Exercice 2/3

3/ Créer une nouvelle colonne, nommée **total**, à la fin de **df_biens** contenant les prix totaux des biens. Renommer la colonne **total** par **prixT**

```
df_biens['total']=df_biens['prix']*df_biens['surface']  
  
df_biens.rename(columns={'total': 'prixT'}, inplace=True)
```

	surface	prix	total		surface	prix	prixT
0	85	120	10200	0	85	120	10200
1	100	200	20000	1	100	200	20000
2	85	150	12750	2	85	150	12750
3	200	180	36000	3	200	180	36000
4	100	420	42000	4	100	420	42000
5	300	330	99000	5	300	330	99000
6	400	400	160000	6	400	400	160000

4/ Créer une nouvelle colonne, nommée **catégorie**, à la fin de **df_biens** qui prend la valeur **'B'** si **prixT** est inférieur ou égal à **32400** et la **surface** est strictement inférieure à **120**, et **'A'** dans le cas contraire

```
df_biens.drop(labels=['surface'], axis=1, inplace=True)
```

	surface	prix	prixT	catégorie
0	85	120	10200	B
1	100	200	20000	A
2	85	150	12750	B
3	200	180	36000	A
4	100	420	42000	A
5	300	330	99000	A
6	400	400	160000	A

Exercice 3/3

5/ Supprimer la colonne **surface** de **df_biens**

```
df_biens.drop(labels=['surface'], axis=1, inplace=True)
```

	prix	prixT	catégorie
0	120	10200	B
1	200	20000	A
2	150	12750	B
3	180	36000	A
4	420	42000	A
5	330	99000	A
6	400	160000	A

6/ Trier les biens de **df_biens** par ordre décroissant selon **prixT** et réinitialiser les indices des lignes

```
df_biens.sort_values(by="prixT", ascending=False,  
                    inplace=True, ignore_index=True)
```

	prix	prixT	catégorie
0	400	160000	A
1	330	99000	A
2	420	42000	A
3	180	36000	A
4	200	20000	A
5	150	12750	B
6	120	10200	B

7/ Afficher le **prix** le plus cher pour chaque catégorie des biens

```
df_biens.groupby(['catégorie'])['prixT'].max()
```

```
catégorie  
A    160000  
B     12750  
Name: prixT, dtype: int64
```

Opérations avancées sur DataFrame (1/4)

● Modification avancée des valeurs d'un DataFrame

```
df['A3']=df['A1'].str.split("-", expand=False)
```

```
df[['A3','A4']]=df['A1'].str.split("-", expand=True)
```

	A1	A2	A3
0	11-aa	32.0	[11, aa]
1	14-bc	20.0	[14, bc]
2	25-dd	NaN	[25, dd]

	A1	A2	A3	A4
0	11-aa	32.0	11	aa
1	14-bc	20.0	14	bc
2	25-dd	NaN	25	dd

df=

	A1	A2
0	11-aa	32.0
1	14-bc	20.0
2	25-dd	NaN

✗ apply vs map

- `DataFrame.apply(func, axis=0` ou `axis=1)` Applique une fonction à chaque colonne ou ligne d'un DataFrame et retourne une Serie
- `DataFrame.map(func)` Applique une fonction élément par élément à chaque valeur d'un DataFrame et retourne un DataFrame
- `Serie.apply(func) ≡ Serie.map(func)` Applique une fonction élément par élément à chaque valeur d'une Series et retourne une nouvelle Series

Opérations avancées sur DataFrame (2/4)

× apply vs map

```
df[["A1","A3"]].apply(lambda x: (x.iloc[0]>10) & (x.iloc[1]!="aa"), axis=1)
```

Serie

```
0    False
1     True
2    False
3    False
dtype: bool
```

df=

	A1	A2	A3
0	30	7	aa
1	50	14	bc
2	20	4	aa
3	8	10	dd

Filtrage des lignes en fonction de certaines conditions

```
df[df[["A1","A3"]].apply(lambda x: (x.iloc[0]>10) & (x.iloc[1]!="aa"), axis=1)]
```

	A1	A2	A3
1	50	14	bc

```
df['A2']=df['A2'].apply(lambda x:df['A1'].min() if x >= 10 else x)
```

Serie

```
0    7
1    8
2    4
3    8
Name: A2, dtype: int64
```

Équivalent à

```
df['A2']=np.where(df['A2']>=10, df['A1'].min(), df['A2'])
```

↑
ndarray

	A1	A2	A3
0	30	7	aa
1	50	8	bc
2	20	4	aa
3	8	8	dd

Opérations avancées sur DataFrame (3/4)

× apply vs map

```
df[["A1","A2"]].map(lambda x: x>=20)
```



DataFrame		
	A1	A2
0	True	False
1	True	False
2	True	False
3	False	False

df=

	A1	A2	A3
0	30	7	aa
1	50	14	bc
2	20	4	aa
3	8	10	dd

● Suppression des lignes en fonction de certaines conditions

```
df.drop(labels=list(df[df['A2'].apply(lambda x: x>10)].index),axis=0,inplace=True)
```

0	False
1	True
2	False
3	False

Name: A2, dtype: bool

	A1	A2	A3
1	50	14	bc

df=

	A1	A2	A3
0	30	7	aa
2	20	4	aa
3	8	10	dd

Opérations avancées sur DataFrame (4/4)

- Indice du maximum `Serie.idxmax()` / `DataFrame.idxmax()`

```
df.idxmax()
```

```
A1    2
A2    1
dtype: int64
```

```
df['A1'].idxmax() #2
```

df=

	A1	A2
0	30	7
1	50	14
2	77	4
3	8	10

- Jointure de deux DataFrames

```
df3=pd.merge(df1, df2, on='A1', how='inner',
```

```
suffixes=["_1", "_2"])
```

```
# par défaut suffixes= ('_x', '_y')
```

df1=

	A1	A2
0	aa	20
1	bc	20
2	aa	20
3	dd	10

df2=

	A1	A2
0	bc	11
1	bc	12
2	wv	14

df3=

	A1	A2_1	A2_2
0	bc	20	11
1	bc	20	12

- Tableau de contingence

```
df4= pd.crosstab(df1["A1"], df1["A2"])
```

df4=

A2	10	20
A1		
aa	0	2
bc	0	1
dd	1	0





PARTIE III

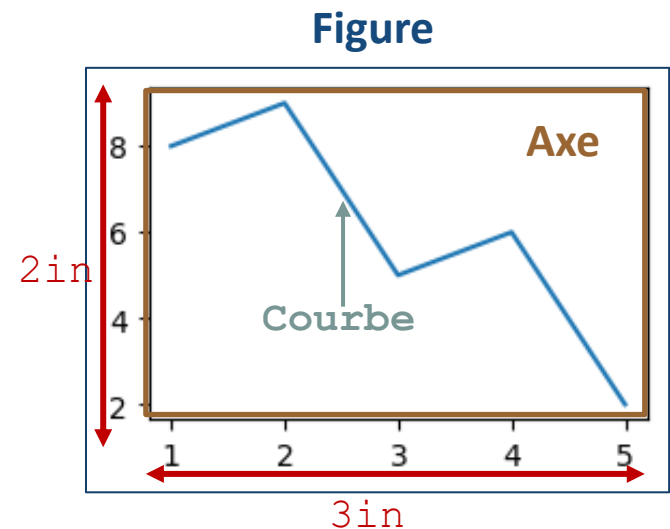
VISUALISATION GRAPHIQUE AVEC `matplotlib` & `seaborn`

Figure 1/2

- **Importation de matplotlib & seaborn :** `import matplotlib.pyplot as plt`
`import seaborn as sns`

- **Traçage et affichage d'une courbe**

```
plt.figure(figsize=(3,2))  
a=range(1,6)  
b=[8, 9, 5, 6, 2]  
plt.plot(a,b) # Traçage  
  
#plt.plot(b) ?  
  
plt.show() # Affichage
```

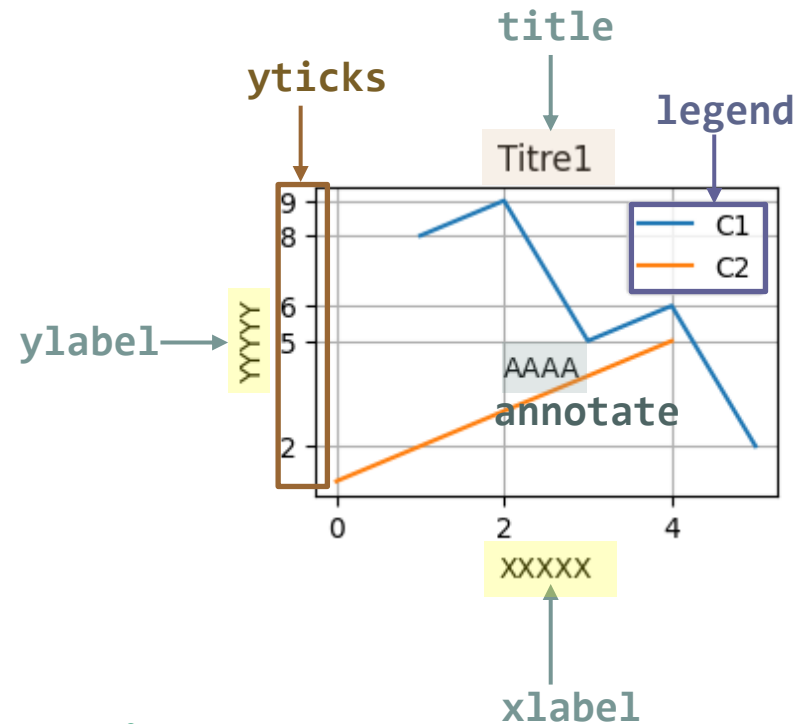


1 in = 2.54 cm (par défaut: [6.4, 4.8])

Figure 2/2

● MAJ des composantes d'une Figure

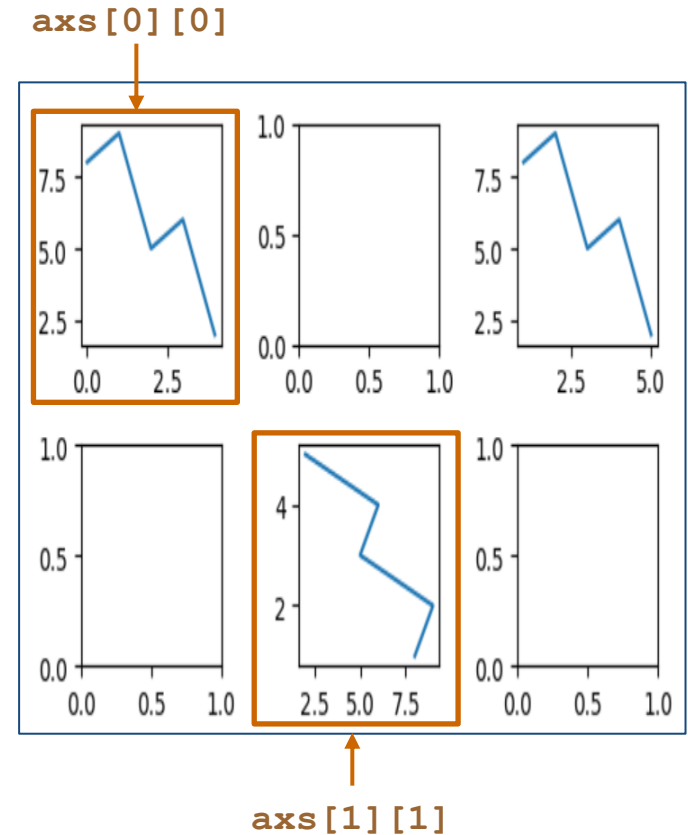
```
plt.figure(figsize=(3,2))
a=range(1,6)
b=[8, 9, 5, 6, 2]
plt.plot(a,b)
plt.plot(a)
plt.title('Titre1')
plt.xlabel('XXXXX')
plt.ylabel('YYYYY')
plt.yticks([8, 9, 5, 6, 2])
plt.annotate('AAAA', xy=(3, 5), xytext=(2, 4))
plt.legend(['C1', 'C2'], loc=0)
plt.grid(True)
plt.show()
```



Sous-Figures (Nombres d'axes ≥ 2) 1/2

```
a=range(1,6)
b=[8, 9, 5, 6, 2]
fig, axs = plt.subplots(2, 3, figsize=(5, 3))
axs[0][0].plot(b)
axs[0][2].plot(a,b)
axs[1][1].plot(b,a)
plt.tight_layout()
plt.show()
```

Ajuster automatiquement les sous-graphiques et leur espacement au sein d'une figure



Sous-Figures (Nombres d'axes ≥ 2) 2/2

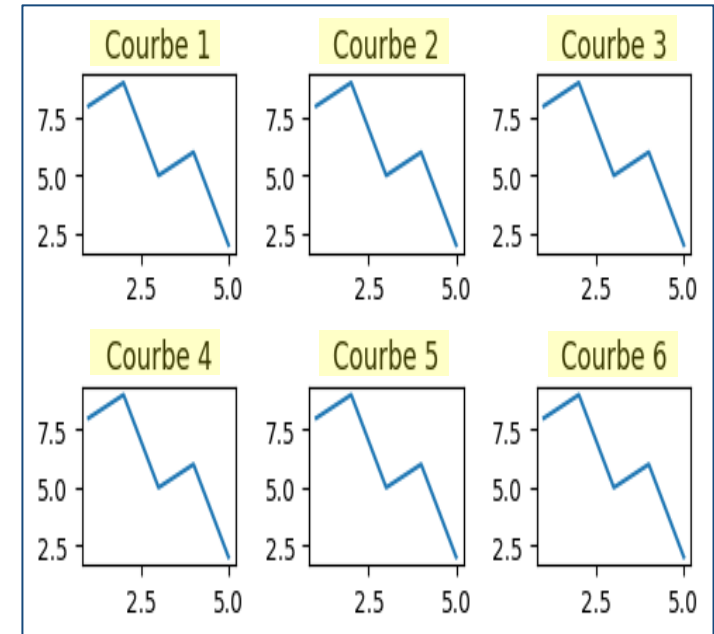
Solution 1

```
a=range(1,6)
b=[8, 9, 5, 6, 2]
fig, axs = plt.subplots(2, 3, figsize=(5, 3))
for i, ax in enumerate(axs.flat):
    ax.plot(a, b)
    ax.set_title("Courbe " + str(i + 1))
plt.tight_layout()
plt.show()
```

Solution 2

```
plt.figure(figsize=(5, 3)) ← Définir la taille
for i in range(6):           totale de la figure
    plt.subplot(2, 3, i + 1)
    plt.plot(a, b)
    plt.title("Courbe " + str(i + 1))
```

Même courbe!

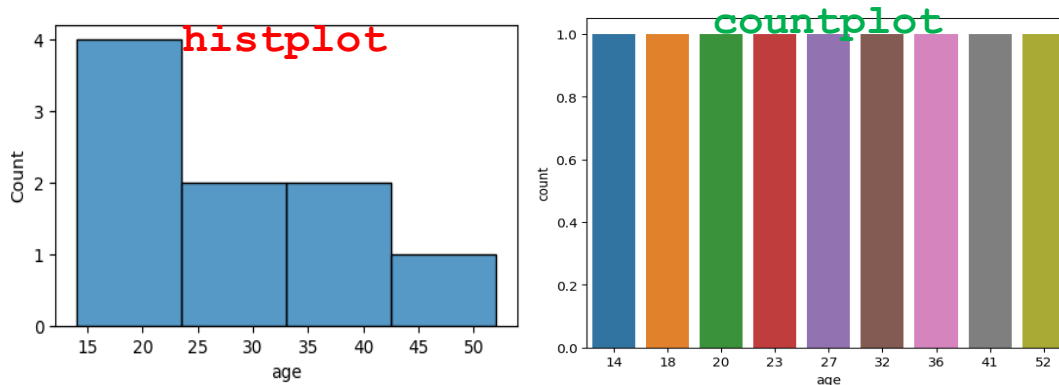


Quelques représentations graphiques (1/3)

● Histogrammes `histplot`/`countplot`

```
#plt.figure(figsize=(5, 3))  
sns.countplot(x="genre", data=df)  
plt.show()
```

```
sns.histplot(x="age", data=df, bins=4)  
plt.show()  
sns.countplot(x="age", data=df)  
plt.show()
```



df=

	patient	age	genre
0	amal	52	femme
1	youssef	14	homme
2	mohamed	36	homme
3	selima	27	femme
4	fatma	18	femme
5	yesmine	20	femme
6	karim	41	homme
7	hedi	32	homme
8	zaynab	23	femme

Quelques représentations graphiques (2/3)

● Diagramme à secteurs (pie chart)

Solution 1

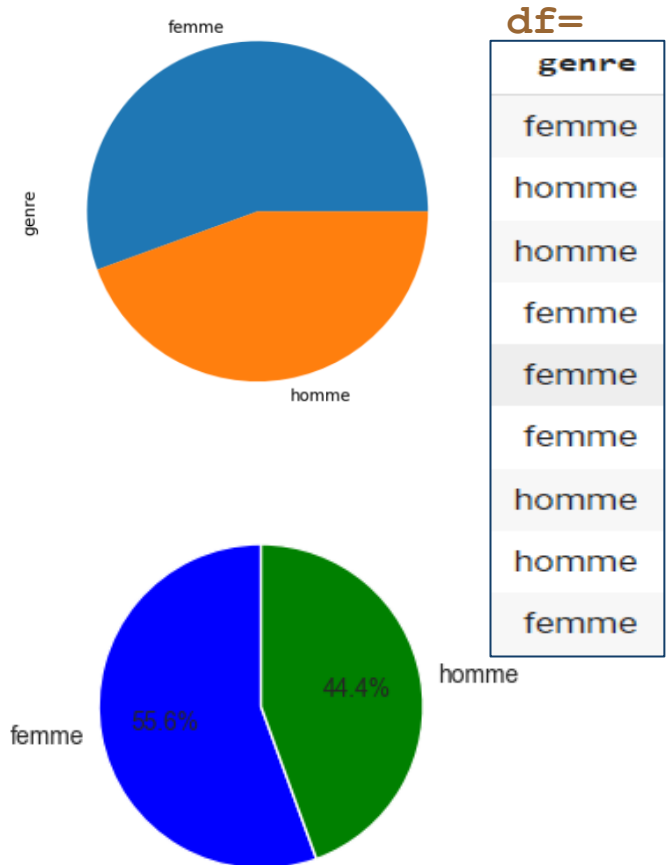
```
#plt.figure(figsize=(5, 3))
```

```
df['genre'].value_counts().plot.pie()
```

```
plt.show()
```

Solution 2

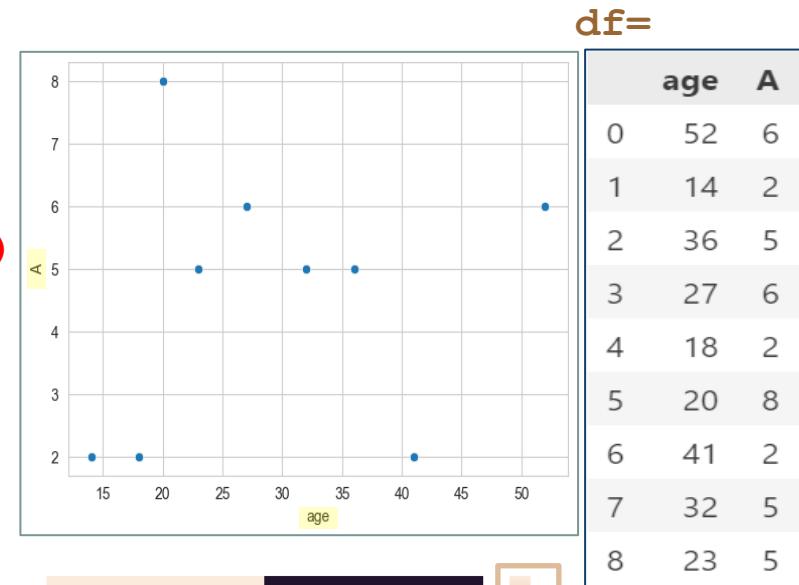
```
genre_counts=df['genre'].value_counts()  
labels=list(genre_counts.index)#[ 'femme','homme']  
colors = ['blue', 'green']  
plt.pie(genre_counts, labels=labels,  
        colors=colors, autopct='%1.1f%%',  
        startangle=90)
```



Quelques représentations graphiques (3/3)

● Nuage de point (scatterplot)

```
#plt.figure(figsize=(5, 3))  
  
sns.scatterplot(data=df, x="age", y="A")  
  
plt.show()
```



● Heatmap (heatmap)

```
#plt.figure(figsize=(5, 3))  
sns.heatmap(df.iloc[:3, [0, 1]],  
            annot=True, fmt='f',  
            cbar=True)  
plt.show()
```

Par défaut `cbar=True`

